Notice that the formula depends directly on the `number_of_shards` variable. That means we cannot change the number of shards once an index is created. If we could change the settings (say, changing the number of shards from two to four), the routing function would break for the existing records, and data wouldn't be found. This is why Elasticsearch doesn't let us change the shards once an index is set up.

What if we have not anticipated data growth, and the shards are exhausted by a spike in data? All is not lost. There's a way out: *reindexing* our data. Reindexing effectively creates a new index with appropriate settings and copies the data from the old index to the new index.

Replicas can be altered on an operational index

While the number of shards cannot be changed when the index is in operation, we can alter the number of replicas if needed. Remember, the routing function is a function of the number of primary shards, not replicas. If we need to change the shard number, we must close the indexes (closed indexes block all read and write operations), change the shard number, and reopen the indexes. Alternatively, we can create a new index with a new set of shards and reindex the data from the old index to the new index.

One of the major goals of Elasticsearch is scalability of the engine. In the next section, we look at Elasticsearch's scalability at a high level: how to scale, how vertical and horizontal scaling work, and the reindexing process. We do not dive into the nitty gritty of scaling solutions here because we come back to this discussion in chapter 14.

3.6 Scaling

When Shay Banon and his team rewrote Elasticsearch from the ground up, one of their goals was ensuring that the server scales effortlessly. That is, if the data increases or the query load escalates, adding additional nodes should solve the problem. Of course, there are other solutions, such as vertical scaling, performance tuning, and more. There are two primary schools of scaling, based on need: scaling horizontally and scaling vertically. Elasticsearch supports both of them.

3.6.1 Scaling up (vertical scaling)

In a vertical scaling scenario, we do not buy additional VMs from our cloud provider but instead add computing resources, such as extra memory, CPU, and I/O, to the existing machines. For example, we may increase the CPU cores, double memory, and so on.

There is also another way to enhance the power of the cluster. Because we now have room, we can install additional nodes on the machine, thus creating multiple nodes in the single (fat) machine.

Remember that scaling up requires the cluster to be shut down, so our application may experience downtime unless we lean on a traditional disaster recovery (DR) model. This brings up our secondary or backup system, which services clients while the primary system is undergoing maintenance.